

Estructuras de Datos

Clase 9 – Mapeos, Diccionarios y Conjuntos



Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Clave

Clave: Atributo de un individuo que sirve para identificarlo en un conjunto de individuos

Ejemplo: DNI, Número de libreta para alumnos.

Ejemplo: Número de afiliado para afiliados a una obra social.

Ejemplo: Número de cuenta para una cuenta de un banco.

Entradas

Problema: ¿Cómo asociar claves con valores?

```
public interface Entry<K,V> {
    public K getKey();           // Retorna la clave de la entrada
    public V getValue();        // Retorna el valor de la entrada
}

public class Entrada<K,V> implements Entry<K,V> {
    private K clave;
    private V valor;

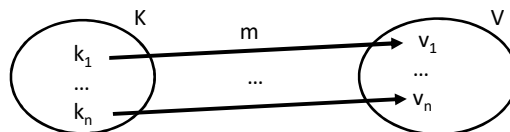
    // Constructor
    public Entrada(K clave, V valor) { this.clave = clave; this.valor = valor; }
    public K getKey() { ... }      // Getters
    public V getValue() { ... }
    public void setKey( K clave ) { ... } // Setters
    public void setValue(V valor) { ... }
    public String toString() {
        return "(" + getKey() + "," + getValue() + ")";
    }
}
```

Estructuras de datos - Dr. Sergio A. Gómez

3

Mapeo

- Un mapeo permite almacenar elementos de tal manera que puedan ser localizados rápidamente usando una clave.
- Motivación: Un elemento generalmente tiene información útil además de su clave.
- Ejemplo: Un alumno tiene clave Libreta pero almacena nombre, fecha de nacimiento, dirección, teléfono, correo electrónico, etc.
- Un mapeo puede ser interpretado como una *función* (o *correspondencia*) de dominio K y codominio V.



- Un mapeo m puede ser interpretado como un conjunto de entradas $m = \{(k_i, v_i), i=1, \dots, n\}$ donde k_i tiene tipo K y v_i tiene tipo V:

$$m = \{ (k_1, v_1), (k_2, v_2), \dots, (k_n, v_n) \}$$

Estructuras de datos - Dr. Sergio A. Gómez

4

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

TDA Mapeo (Map)

Dado un mapeo M:

- **size():** Retorna el número de entradas de M
- **isEmpty():** Testea si M es vacío
- **get(k):**
 - Si M contiene una entrada e con clave igual a k, retorna el valor de v;
 - sino retorna null
- **put(k,v):**
 - Si M no tiene una entrada con clave k, entonces agrega una entrada (k,v) a M y retorna null
 - Si M ya tiene una entrada e con clave k, reemplaza el valor con v en e y retorna el valor viejo de e.

Estructuras de datos - Dr. Sergio A. Gómez

5

- **remove(k):**
 - Remueve de M la entrada con clave k y retorna su valor
 - Si M no tiene entrada con clave k, retorna null.
- **keys():**
 - Retorna una colección iterable de las claves en M.
 - `keys().iterator()`: retorna un iterador de claves.
- **values():**
 - Retorna una colección iterable con los valores de las claves almacenadas en M
 - `values().iterator()`: retorna un iterador de valores
- **entries():**
 - Retorna una colección iterable con las entradas de M.
 - `entries().iterator()` retorna un iterador de entradas.

Estructuras de datos - Dr. Sergio A. Gómez

6

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Ejemplo

tiempo	Operación	Salida	Mapeo M de enteros en caracteres
1	isEmpty()	true	$\emptyset$
2	put(5, A)	null	{(5,A)}
3	put(7,B)	null	{(5,A), (7,B)}
4	put(2,C)	null	{(5,A), (7,B), (2,C)}
5	put(8,D)	null	{(5,A), (7,B), (2,C), (8,D)}
6	put(2,E)	C	{(5,A), (7,B), (2,E), (8,D)}
7	get(7)	B	{(5,A), (7,B), (2,E), (8,D)}
8	get(4)	null	{(5,A), (7,B), (2,E), (8,D)}
9	get(2)	E	{(5,A), (7,B), (2,E), (8,D)}
10	size()	4	{(5,A), (7,B), (2,E), (8,D)}
11	remove(5)	A	{(7,B), (2,E), (8,D)}
12	remove(2)	E	{(7,B), (8,D)}
13	get(2)	null	{(7,B), (8,D)}
14	isEmpty()	false	{(7,B), (8,D)}

Estructuras de datos - Dr. Sergio A. Gómez

7

Interfaz Map en Java

```

public interface Map<K,V> {
    public int size();
    public boolean isEmpty();
    public V get(K key);
    public V put(K key, V value);
    public V remove(K key);
    public Iterable<K> keys();
    public Iterable<V> values();
    public Iterable<Entry<K,V>> entries();
}

```

Estructuras de datos - Dr. Sergio A. Gómez

8

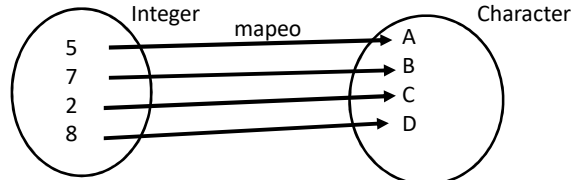
El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Ejemplo de uso del mapeo en Java

Creo un mapeo de enteros en caracteres llamado *mapeo*:

```
Map<Integer, Character> mapeo = new MapeoConLista<Integer, Character>();
boolean vacio = mapeo.isEmpty();    vacío = true porque el mapeo no tiene entradas
mapeo.put( 5, 'A' );
vacio = mapeo.isEmpty();    vacío = false porque el mapeo tiene al menos una entrada
mapeo.put( 7, 'B' );
mapeo.put( 2, 'C' );
mapeo.put( 8, 'D' );
Character c = mapeo.get(7);    c vale B porque (7,B) está en el mapeo
Character d = mapeo.get(9);    d vale null porque no hay entrada con clave 9 en el mapeo

for( Entry<Integer,Character> e : mapeo.entries()) { Muestra (5,A), (7,B), (2,C), (8,D)
    System.out.println("Entrada: " + e);
}
```



Estructuras de datos - Dr. Sergio A. Gómez

9

Implementación de Mapeo con Lista: Estructura de datos y Constructor

Implementamos el mapeo con una clase genérica *MapeoConLista* que recibe el tipo de las claves K y el tipo de los valores V de las entradas.

La clase *MapeoConLista* implementa la interfaz *Map*.

```
public class MapeoConLista<K, V> implements Map<K, V> {
```

El estado interno del mapeo está representado con una lista S doblemente enlazada de entradas (clave,valor) donde clave tiene tipo K y valor tiene tipo V:

```
protected ListaDoblementeEnlazada<Entrada<K,V>> S;
```

El constructor del mapeo crea una lista de entradas vacía:

```
public MapeoConLista() {
    S = new ListaDoblementeEnlazada<Entrada<K,V>>();
}
```

Estructuras de datos - Dr. Sergio A. Gómez

10

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Implementación de Mapeo con Lista: put

put(key, value): Si M no tiene una entrada con clave key, entonces
 agrega una entrada (key,value) a M y retorna null
 Si M ya tiene una entrada e con clave key,
 reemplaza el valor con value en e y retorna el valor viejo de e.

```
public V put( K key, V value ) {
    // Para cada posición p de la lista S hacer:
    for( Position<Entrada<K,V>> p : S.positions() )
        // Si la clave de la entrada en la posición p es key:
        if( p.element().getKey().equals( key ) ) {
            // Salvar el valor de la entrada en aux
            V aux = p.element().getValue();
            // Setear el nuevo valor de la entrada a value
            p.element().setValue( value );
            // Retornar el viejo valor
            return aux;
        }
    // Si salí del for-each entonces no encontré una entrada con clave key
    S.addLast(new Entrada<K,V>(key, value) ); // Inserto una nueva entrada (key,value)
    return null; // Retorno null para indicar que inserté una nueva entrada
}
```

Complejidad:
 Si el mapeo tiene n
 entradas, el peor
 escenario es cuando k no
 está en S, entonces
 $T_{\text{put}}(n) = O(n)$
Tarea: Reimplementar
 con while y bandera de
 corte (en siguiente diapo)

Estructuras de datos - Dr. Sergio A. Gómez

11

Implementación de Mapeo con Lista: put estructurado

put(key, value): Si M no tiene una entrada con clave key, entonces
 agrega una entrada (key,value) a M y retorna null
 Si M ya tiene una entrada e con clave key,
 reemplaza el valor con value en e y retorna el valor viejo de e.

```
public V put( K key, V value ) {
    // Para cada posición p de la lista S hacer:
    Iterator<Position<Entrada<K,V>> it = S.positions().iterator();
    V resultado = null;
    while( it.hasNext() && resultado == null )
        // Si la clave de la entrada en la posición p es key:
        Position<Entrada<K,V>> p = it.next();
        if( p.element().getKey().equals( key ) ) {
            // Salvar el valor de la entrada en resultado
            resultado = p.element().getValue();
            // Setear el nuevo valor de la entrada a value
            p.element().setValue( value );
        }
    }
    if ( resultado == null ) S.addLast(new Entrada<K,V>(key, value) ); // Inserto una
    nueva entrada (key,value)
    return resultado;
}
```

Estructuras de datos - Dr. Sergio A. Gómez

12

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Implementación de Mapeo con Lista: get

get(key): Retorna el valor para la clave key en M si hay una entrada con clave key o null si no hay entrada con clave key.

```
public V get(K key) {
    // Para cada posición p de la lista S hacer:
    for( Position<Entrada<K,V>> p : S.positions() )
        // Si la clave de la entrada actual es key:
        if( p.element().getKey().equals( key ) )
            // Retornar el valor de la entrada actual:
            return p.element().getValue();
    // Si salí del for-each, quiere decir que no encontré ninguna entrada con clave key.
    return null;
}
```

Nota: En el peor escenario se recorre toda la lista y se retorna null. Sin embargo, el código se puede hacer más eficiente porque positions recorre toda la lista cuando no debería ser necesario.

Tarea: Reescribir el código para iterar S y no S.positions(). Pedir un iterador a la lista S y usar while con bandera de corte.

Estructuras de datos - Dr. Sergio A. Gómez

13

Implementación de Mapeo con Lista: remove

remove(k): Retorna el valor removido value para (key,value) o null si key no está en M

```
public V remove(K key ) {
    // Para cada posición p de S hacer:
    for( Position<Entrada<K,V>> p : S.positions() )
        // Si la entrada de la posición p tiene clave key:
        if( p.element().getKey().equals( key ) ) {
            // Salvar el valor de la entrada corriente en value:
            V value = p.element().getValue();
            // Eliminar la posición p de la lista S:
            S.remove( p );
            // Retornar el valor de entrada removida del mapeo:
            return value;
        }
    // Si salí del for-each, quiere decir que no encontré ninguna entrada con clave key
    return null;
}
```

Tarea: Reprogramar pidiendo iterador de posiciones y usar while con bandera de corte.

Estructuras de datos - Dr. Sergio A. Gómez

14

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Aplicación

- Problema: Dada una cola Q de caracteres, determinar la frecuencia de apariciones de cada carácter.
- Nota: La frecuencia de apariciones de un carácter C en una secuencia de caracteres Q se define como la cantidad de apariciones de C dividido la cantidad total de caracteres de Q.
- Ejemplo: Si 'R' apareció 4 veces y hay 200 caracteres en la secuencia Q, entonces la frecuencia de apariciones de 'R' en Q es:

$$4/200 = 0,02 = 2\%.$$

Estructuras de datos - Dr. Sergio A. Gómez

15

```

public static Map<Character, Float> calcularFrecuenciaApariciones(Queue<Character> q) {
    ... // Agregar todos los try-catch que falten
    Map<Character, Integer> m = new MapeoConLista<Character, Integer>();
    int cantidadTotalCaracteres = q.size(); // Salvo el largo de q
    while( !q.isEmpty() ) { // mientras la cola q no se vacíe
        char c = q.dequeue(); // obtener un elemento c de la cola q
        if( m.get(c) == null ) // Si es la primera vez que veo a c
            m.put( c, 1 ); // Anoto que c apareció exactamente 1 vez
        else
            m.put( c, m.get(c) + 1 ); // Anoto que c apareció una vez más
    }

    // Armo un mapeo nuevo para retornar las frecuencias:
    Map<Character, Float> resultado = new MapeoConLista<Character, Float>();

    // Para cada entrada e de m hacer:
    for( Entry<Character, Integer> e : m.entries() )
        // La frecuencia de e.getKey es e.getValue / largo original de q
        // Casteo entero a real para forzar a que / se interprete como división real:
        resultado.put(e.getKey(), ((float) e.getValue()) / cantidadTotalCaracteres );

    return resultado; // Retorno el mapeo resultado
}

```

Estructuras de datos - Dr. Sergio A. Gómez

16

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Diccionarios

- Un diccionario almacena pares clave-valor (k,v) (como los mapeos)
- Las claves son de tipo K y los valores de tipo V genéricos (como los mapeos)
- (A diferencia de los mapeos) un diccionario puede almacenar claves repetidas.
- Hay dos tipos de diccionarios:
 - Diccionarios no ordenados: Las claves sólo se comparan por igual.
 - Diccionarios ordenados: Hay una relación de orden total definida para las claves.

Nota: Definición intuitiva de *orden total*: todas las claves son comparables entre sí por $\leq$.

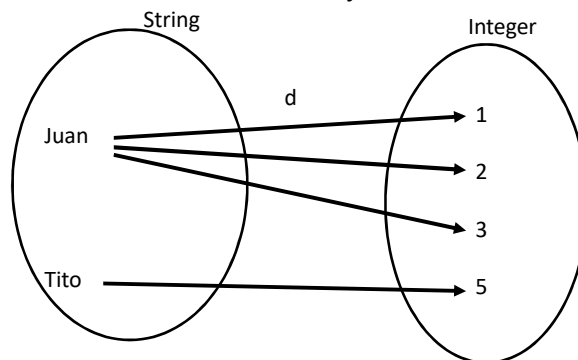
Abstraeremos la relación $a \leq b$ con `a.compareTo(b)` y con `comp.compare(a,b)` si `comp` es de tipo `Comparator`.

Estructuras de datos - Dr. Sergio A. Gómez

17

Interpretación matemática del diccionario

Un diccionario puede ser interpretado como un mapeo multivaluado o como una relación entre 2 conjuntos.



En esta clase 8, representaremos al diccionario `d` con el conjunto de entradas:

$\{ (Juan, 1), (Juan, 2), (Juan, 3), (Tito, 5) \}$

Estructuras de datos - Dr. Sergio A. Gómez

18

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

ADT Diccionario

Dado un diccionario no ordenado D :

- **size()**: Retorna el número de entradas de D .
- **isEmpty()**: Testea si D está vacío.
- **find(key)**: Si D contiene una entrada e con clave igual a key , entonces retorna e , sino retorna *null*.
- **findAll(key)**: Retorna una colección iterable conteniendo todas las entradas con clave igual a key .
- **insert(key,value)**: Inserta en D una entrada e con clave key y valor $value$ y retorna la entrada e .
- **remove(e)**: Remueve de D la entrada e , retornando la entrada removida; ocurre un error si e no está en D .
- **entries()**: Retorna una colección iterable con las entradas clave-valor de D .

Estructuras de datos - Dr. Sergio A. Gómez

19

Ejemplo

Tiempo	Operación	Salida	Diccionario de enteros en caracteres
1	insert(5,A)	(5,A)	{(5,A)}
2	insert(7,B)	(7,B)	{(5,A), (7,B)}
3	insert(2,C)	(2,C)	{(5,A), (7,B), (2,C)}
4	insert(8,D)	(8,D)	{(5,A), (7,B), (2,C), (8,D)}
5	insert(2,E)	(2,E)	{(5,A), (7,B), (2,C), (8,D), (2,E)}
6	find(7)	(7,B)	{(5,A), (7,B), (2,C), (8,D), (2,E)}
7	find(4)	null	{(5,A), (7,B), (2,C), (8,D), (2,E)}
8	find(2)	(2,C)	{(5,A), (7,B), (2,C), (8,D), (2,E)}
9	findAll(2)	{(2,C), (2,E)}	{(5,A), (7,B), (2,C), (8,D), (2,E)}
10	size()	5	{(5,A), (7,B), (2,C), (8,D), (2,E)}
11	remove(find(5))	(5,A)	{(7,B), (2,C), (8,D), (2,E)}
12	find(5)	null	{(7,B), (2,C), (8,D), (2,E)}

Estructuras de datos - Dr. Sergio A. Gómez

20

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Interfaz Dictionary

```

public interface Dictionary<K,V> {
    // Retorna el número de entradas de D
    public int size();

    // Testea si D está vacío
    public boolean isEmpty();

    // Si D contiene una entrada e con clave igual a key,
    // entonces retorna e, sino retorna null
    public Entry<K,V> find(K key);

    // Retorna una colección iterable conteniendo
    // todas las entradas con clave igual a key
    public Iterable<Entry<K,V>> findAll(K key);

    // insert(k,v): Inserta en D una entrada e con clave key y valor value y retorna la entrada e
    public Entry<K,V> insert(K key, V value);

    // remove(e): Remueve de D una entrada e, retornando la entrada removida;
    // ocurre un error si e no está en D
    public Entry<K,V> remove( Entry<K,V> e ) throws NonExistentEntryException;

    // entries(): Retorna una colección iterable con las entradas clave-valor de D.
    public Iterable<Entry<K,V>> entries();
}

```

Estructuras de datos - Dr. Sergio A. Gómez

21

Implementación de Diccionarios no ordenados con Lista

```

public class DiccionarioConLista<K,V> implements Dictionary<K,V> {
    protected ListaDoblementeEnlazada<Entry<K,V>> D;

    public DiccionarioConLista() { D = new ListaDoblementeEnlazada<Entry<K,V>>(); }
    public Iterable<Entry<K,V>> entries() { return D; }

    // Agrega una entrada (key,value) a D y la devuelve
    public Entry<K,V> insert(K key, V value ) {
        Entry<K,V> e = new Entrada<K,V>(key, value);
        D.addLast( e ); // Asumo que addLast funciona en orden 1.
        return e;
    }

    // remove(e): Remueve la entrada e del diccionario y retorna la entrada removida.
    public Entry<K,V> remove( Entry<K,V> e ) throws NonExistentEntryException {
        for( Position<Entry<K,V>> p : D.positions() ) {
            if( p.element() == e ) { D.remove(p); return e; }
        }
        throw new NonExistentEntryException( "Diccionario::remove: ..." );
    }
}

```

Estructuras de datos - Dr. Sergio A. Gómez

22

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Implementación de Diccionarios no ordenados con Lista (cont)

```
public Iterable<Entry<K,V>> findAll( K key ) {
    PositionList<Entry<K,V>> l =
        new ListaDoblementeEnlazada<Entry<K,V>>();
    for( Position<Entry<K,V>> p : D.positions() )
        if( p.element().getKey().equals(key) )
            l.addLast( p.element() );
    return l;
}
```

Note que al usar positions() hay una pasada inicial sobre la lista D.
Se puede evitar usando las posiciones directamente con first, last y next.

Conjuntos

Un conjunto, caracterizado con el tipo Set<E>, tiene definidas las operaciones:

- insert(x): Inserta un elemento x al conjunto
- delete(x): Elimina el elemento x del conjunto
- member(x): Retorna true si x pertenece al conjunto
- intersection(S): Interseca al conjunto con otro conjunto S y retorna un nuevo conjunto intersección
- union(S): Une al conjunto con el conjunto S y retorna un nuevo conjunto union
- complement(): Calcula y retorna el complemento del conjunto (solo se puede implementar si el dominio es finito)
- iterator(): Retorna un iterador con los elementos del conjunto

Conjuntos: Implementación

Usando una lista doblemente enlazada:

- `insert(x)`: insertar x al principio de la lista
- `delete(x)`: Buscar x en la lista y eliminarlo
- `member(x)`: Buscar x en la lista
- `intersection(S)`: por cada elemento x de this buscarlo en S y si está en S insertarlo en el resultado.
- `union(S)`: cada elemento de this se inserta en el resultado y cada elemento de S si no está en el resultado se inserta en el resultado.
- `complement()`: para cada elemento del dominio si no está en this se agrega al resultado.
- `iterator()`: Retornar un iterador para la lista subyacente

Tarea: Implementar la interfaz `Set<E>` usando como representación interna:

(1) una lista enlazada de E's y (2) un mapeo de E en Boolean.

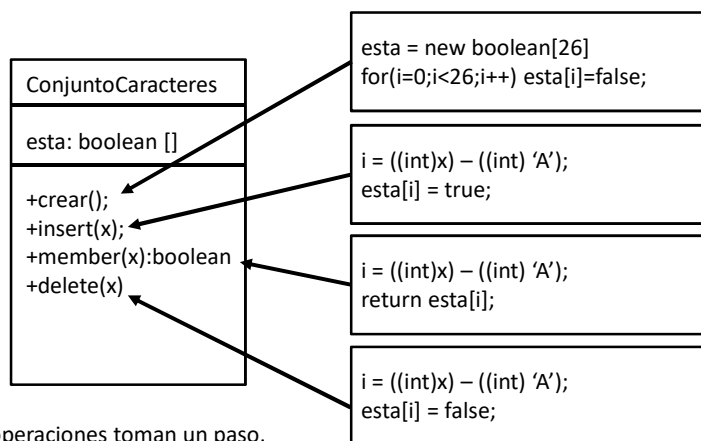
Estructuras de datos - Dr. Sergio A. Gómez

25

Conjuntos con dominios finitos: arreglos de bits

Supongamos que queremos representar un conjunto de caracteres formado por caracteres de A a Z.

El estado interno del conjunto es un arreglo de 26 booleanos uno para cada letra c, indicando si c está o no en el conjunto.



Las operaciones toman un paso.

Estructuras de datos - Dr. Sergio A. Gómez

26

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Ejercicio: Tipo Bag (bolsa)

Un bag es una colección que soporta elementos repetidos:

Interfaz Bag<E> extiende a Iterable<E>:

void add(E item): *inserta un ítem*

boolean isEmpty(): *¿está la bolsa vacía?*

int size(): *número de elementos en la bolsa*

void remove(E item): *elimina una aparición de ítem en la bolsa*

int getCount(E ítem): *cuenta cuántas veces está ítem en la bolsa*

Implementar la interfaz Bag usando dos representaciones posibles: (1) una lista doblemente enlazada y (2) un diccionario.

Estructuras de datos - Dr. Sergio A. Gómez

27

Bibliografía

- Capítulo 9 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.

Estructuras de datos - Dr. Sergio A. Gómez

28

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.